Program Studi Teknik Informatika
Institut Teknologi Sumatera

MODUL PRAKTIKUM

Sistem Operasi

IF25-12007 | Genap 2025/2026

Mata Kuliah	Sistem Operasi (IF25-12007)
Semester	Genap 2025/2026
Institusi	Institut Teknologi Sumatera

© 2026 — Institut Teknologi Sumatera

TIM PENGAJAR & ASISTEN

Dosen Pengampu

Martin Clinton Tosima Manullang, S.T., M.T., Ph.D.

Ilham Firman Ashari, S.Kom., M.T

I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.

Eko Dwi Nugroho, S.Kom., M.Cs.

Asisten Praktikum

Ahmat Prayoga Sembiring	123140053
Andryano Shevchenko Limbong	123140205
Aziz Kurniawan	122140097
Dito Rifki Irawan	122140153
Ebentua Philippus Limbong	123140086
Elsa Elisa Yohana Sianturi	122140135
Fathan Andi Kartagama	122140055
Miftahul Khoiriyah	123140064
Muhammad Bintang Al-Fasya	123140098
Muhammad Fadhil Zurani	122140146
Reza Chairul Manam	120140086
Stevanus Cahya Anggara	123140038
Varasina Farmadani	123140107
Zakhi Algifari	122140198

Contents

1 Sinkronisasi: Race Condition dan Mutex/Lock 4

1.1 Tujuan 4

1.2 Sesi 1 — Menjalankan Demo Race Condition. 4

1.2.1 Persiapan dan Kompilasi 4

1.2.2 Analisis Kode Penyebab Race Condition 5

1.3 Sesi 2 — Implementasi *Semaphore* 6

1.3.1 Struktur *Semaphore* pada implementasi OS 6

1.3.2 Implementasi sema_down() dan sema_up() 7

1.4 Sesi 3 — Implementasi *Lock* dan Integrasi Kernel 7

1.4.1 Implementasi lock_acquire() dan lock_release() 8

1.4.2 Aktivasi Sinkronisasi dan Analisis *Critical Section* 9

1.5 Sesi 4 — Kompilasi dan Verifikasi Sistem 10

1.6 Eksplorasi Mandiri 11

CHAPTER 1

Sinkronisasi: Race Condition dan Mutex/Lock

1.1 Tujuan

Kerjakan tugas-tugas berikut secara berurutan mengikuti alur sesi pada Modul 9 dan Modul 10. Sertakan bukti berupa tangkapan layar (*screenshot*) QEMU atau penggalan kode yang relevan pada setiap langkah yang diminta.

1.2 Sesi 1 — Menjalankan Demo Race Condition

1.2.1 Persiapan dan Kompilasi

Pastikan implementasi Minggu 4 sudah berfungsi, lalu kompilasi dengan *flag* sinkronisasi diaktifkan.

1. Jalankan perintah berikut untuk mengompilasi dan menjalankan kernel dengan *flag* `SYNCHRONIZATION`:

```
make clean
make DEMO=-DSYNCHRONIZATION
make run

```

Kode 1.1: Compile dan jalankan demo race condition

Amati keluaran (*output*) yang muncul selama ±15 detik, lalu ambil tangkapan layar (*screenshot*) pada layar QEMU. Tempel hasil tangkapan layar di sini:

[Tempel screenshot layar di sini]

Gambar 1.1: Tampilan QEMU pada kondisi *race condition* tanpa sinkronisasi.

2. Perhatikan output yang muncul. Isilah tabel berikut berdasarkan anomali yang kamu temukan pada layar QEMU.

Tabel 1.1: Anomali yang ditemukan pada output race condition

No.	Anomali yang Terlihat	Kemungkinan Penyebab
1	*[contoh: baris terpotong di tengah]*	*[Isi jawaban di sini]*
2	*[Isi jawaban di sini]*	*[Isi jawaban di sini]*
3	*[Isi jawaban di sini]*	*[Isi jawaban di sini]*

1.2.2 Analisis Kode Penyebab Race Condition

1. Buka file `src/kernel.c` dan perhatikan fungsi `producer()` dan `consumer()`. Isilah tabel berikut untuk mengidentifikasi letak jebakan `thread_yield()` pada masing-masing fungsi.
2. Tuliskan satu skenario eksekusi konkret yang menjelaskan mengapa angka yang dicetak *Consumer* bisa berbeda dari yang ditulis *Producer*. Ikuti format langkah demi langkah berikut:

1.3 Sesi 2 — Implementasi *Semaphore*

Tabel 1.2: Letak `thread_yield()` yang berbahaya pada *critical section*

Fungsi	Posisi thread_yield()	Akibat jika CPU berpindah di sini
`producer()`	Setelah `shared_buffer = item`, sebelum mencetak log	*[Isi jawaban di sini]*
`consumer()`	Setelah mencetak `"membaca item "`, sebelum `putint()`	*[Isi jawaban di sini]*

Tabel 1.3: Skenario *race condition* langkah demi langkah

Langkah	Thread Aktif	Yang Terjadi
1	*Producer*	`shared_buffer = 3`
2	*Producer*	`thread_yield()` dipanggil — CPU berpindah
3	*Consumer*	*[Isi jawaban di sini]*
4	*Consumer*	*[Isi jawaban di sini]*
5	*Producer*	*[Isi jawaban di sini]*
6	*Consumer*	*[Isi: angka yang tercetak = ?]*

1.3.1 Struktur *Semaphore* pada implementasi OS

1. Perhatikan definisi `struct semaphore` di `src/synch.h`. Isilah tabel berikut berdasarkan kode sumber.

Tabel 1.4: Field-field pada `struct semaphore`

Field	Tipe Data	Fungsi
`value`	`unsigned`	*[Isi jawaban di sini]*
`waiters[]`	`struct thread *[]`	*[Isi jawaban di sini]*
`waiter_count`	`uint32_t`	*[Isi jawaban di sini]*

2. Perhatikan nilai awal *semaphore* yang diinisialisasi di `kernel_main()` pada `kernel.c`:

```
sema_init(&sema_empty, 1);
sema_init(&sema_full,  0);

```

Kode 1.2: Inisialisasi *semaphore* di `kernel_main()`

Isilah tabel berikut untuk menjelaskan mengapa nilai awal kedua *semaphore* berbeda.

Tabel 1.5: Alasan nilai awal *semaphore*

Semaphore	Nilai Awal	Alasan
`sema_empty`	1	*[Jelaskan mengapa bernilai 1]*
`sema_full`	0	*[Jelaskan mengapa bernilai 0]*

1.3.2 Implementasi sema_down() dan sema_up()

1. Salin kode yang telah diimplementasikan di sini:

```c
void sema_down(struct semaphore *sema) {

    /* Tulis implementasi Anda di sini */

}

```

Kode 1.3: Implementasi `sema_down()` oleh mahasiswa

```c
void sema_up(struct semaphore *sema) {

    /* Tulis implementasi Anda di sini */

}

```

Kode 1.4: Implementasi `sema_up()` oleh mahasiswa

2. Jelaskan mengapa `sema_down()` menggunakan `while` dan bukan `if` untuk mengecek `sema->value == 0`. Apa yang dapat terjadi jika menggunakan `if`?

   Jawaban:
   [Isi jawaban di sini]

1.4 Sesi 3 — Implementasi *Lock* dan Integrasi Kernel

1.4.1 Implementasi lock_acquire() dan lock_release()

1. Masih di `src/synch.c`, implementasikan `lock_acquire()` dan `lock_release()`. Salin kode yang telah diimplementasikan di sini:

```
void lock_acquire(struct lock *lock) {

    /* Tulis implementasi Anda di sini */

}

```

Kode 1.5: Implementasi `lock_acquire()` oleh mahasiswa

```
void lock_release(struct lock *lock) {

    /* Tulis implementasi Anda di sini */

}

```

Kode 1.6: Implementasi `lock_release()` oleh mahasiswa

2. Isilah tabel berikut untuk menjelaskan peran *field* `holder` pada `struct lock`.

Tabel 1.6: Peran *field* `holder` pada `struct lock`

Pertanyaan	Jawaban
Apa yang disimpan di `holder`?	*[Isi jawaban di sini]*
Kapan `holder` diisi (*non-NULL*)?	*[Isi jawaban di sini]*
Kapan `holder` dikosongkan (`NULL`)?	*[Isi jawaban di sini]*
Apa akibatnya jika `holder` tidak pernah dibersihkan (*reset*) ke `NULL`?	*[Isi jawaban di sini]*

1.4.2 Aktivasi Sinkronisasi dan Analisis *Critical Section*

1. Buka `src/kernel.c`. Hilangkan tanda komentar (*uncomment*) pada keempat blok `TODO` di fungsi `producer()` dan `consumer()` untuk mengaktifkan pengaman. Perhatikan bahwa kini area manipulasi data telah diapit oleh fungsi sinkronisasi (membentuk *Entry Section* dan *Exit Section*).

```c
static void producer(void *aux) {
    (void)aux;
    int item = 0;
    while (1) {
        sema_down(&sema_empty);   /* <- sudah di-uncomment */
        lock_acquire(&buf_lock);  /* <- sudah di-uncomment */

        /* --- CRITICAL SECTION --- */
        shared_buffer = item;
        thread_yield();
        vga_puts("[Producer] menulis item ");
        vga_putint(item++);
        vga_putchar('\n');
        /* --- END CRITICAL SECTION --- */

        lock_release(&buf_lock);  /* <- sudah di-uncomment */
        sema_up(&sema_full);      /* <- sudah di-uncomment */
        thread_yield();
    }
}

```

Kode 1.7: Tampilan `producer()` setelah hilangkan blok komentar

2. Secara teoritis, *Mutual Exclusion* (*Mutex*) digunakan untuk melindungi variabel global di memori (dalam hal ini `shared_buffer`). Namun pada kode di atas, instruksi pencetakan ke layar (`vga_puts` dan `vga_putint`) juga ikut dimasukkan ke dalam area *Critical Section* yang digembok oleh `buf_lock`.

   Analisislah mengapa perangkat antarmuka/layar (VGA) juga dikategorikan sebagai *shared resource* dalam sebuah Sistem Operasi! Apa yang akan

terjadi pada integritas data dan tampilan di layar jika `lock_release(&buf_lock)` dipindahkan ke atas, tepat setelah baris `shared_buffer = item;` (sebelum mencetak log)?

Jawaban:
[Isi jawaban di sini]

1.5 Sesi 4 — Kompilasi dan Verifikasi Sistem

1. *Compile* ulang dan jalankan kernel dengan sinkronisasi penuh:

```
make clean
make DEMO=-DSYNCHRONIZATION
make run

```

Kode 1.8: Compile dan jalankan dengan sinkronisasi penuh

Sertakan tangkapan layar (*screenshot*) keluaran (*output*) QEMU **sebelum** dan **sesudah** sinkronisasi diterapkan secara berdampingan.

[Tempel screenshot layar di sini]

(a) Sebelum sinkronisasi (*race condition*)

[Tempel screenshot layar di sini]

(b) Setelah sinkronisasi

Gambar 1.2: Perbandingan tampilan QEMU sebelum dan sesudah penerapan mekanisme sinkronisasi.

2. Verifikasi urutan keluaran (*output*) yang benar. Ekspektasi keluaran (*output*) setelah sinkronisasi berhasil adalah sebagai berikut:

```
[Producer] menulis item 0
[Consumer] membaca item 0
[Producer] menulis item 1
[Consumer] membaca item 1
[Producer] menulis item 2
[Consumer] membaca item 2
...

```

Kode 1.9: Ekspektasi keluaran (*output*) *Producer-Consumer* yang tersinkronisasi

3. Apakah keluaran (*output*) pada QEMU sudah konsisten seperti di atas? Jelaskan mengapa `thread_yield()` yang ada di dalam *critical section* tidak lagi menyebabkan anomali setelah `lock` dipasang.

   Jawaban:
   [Isi jawaban di sini]

4. Isilah tabel ringkasan peran ketiga instrumen sinkronisasi berdasarkan hasil praktikum yang telah dilakukan.

Tabel 1.7: Peran tiga instrumen sinkronisasi pada *Producer-Consumer*

Instrumen	Jenis	Peran
`sema_empty`	*Counting Semaphore*	*[Jelaskan kapan Producer harus menunggu]*
`sema_full`	*Counting Semaphore*	*[Jelaskan kapan Consumer harus menunggu]*
`buf_lock`	*Mutex Lock*	*[Jelaskan apa yang dilindungi]*

1.6 Eksplorasi Mandiri

1. **Eksperimen tanpa *Lock*.**
   Pada `kernel.c`, jadikan blok komentar (*comment out*) untuk pemanggilan `lock_acquire()` dan `lock_release()` pada `producer()` dan `consumer()`, tetapi biarkan `sema_down()` dan `sema_up()` tetap aktif. *Compile* ulang dan jalankan.

```
make clean && make DEMO=-DSYNCHRONIZATION && make run

```

Kode 1.10: Kompilasi untuk eksperimen tanpa *lock*

(a) Sertakan tangkapan layar (*screenshot*) hasil QEMU dari eksperimen ini.

[Tempel screenshot layar di sini]

Gambar 1.3: Tampilan QEMU tanpa `lock` (semaphore tetap aktif).

(b) Apakah keluaran (*output*) masih kacau atau sudah rapi? Jelaskan mengapa.

Jawaban:
[Isi jawaban di sini]

2. **Eksperimen tanpa *Semaphore*.**
Kembalikan `lock_acquire()` dan `lock_release()` seperti semula, lalu jadikan blok komentar (*comment out*) untuk pemanggilan semua `sema_down()` dan `sema_up()`. *Compile* ulang dan jalankan.

(a) Sertakan tangkapan layar (*screenshot*) hasil QEMU dari eksperimen ini.

[Tempel screenshot layar di sini]

Gambar 1.4: Tampilam QEMU tanpa *semaphore* (`lock` tetap aktif).

(b) Apakah keluaran (*output*) masih konsisten? Apa perbedaan yang dapat diamati dibandingkan saat ketiganya aktif? Jelaskan.

Jawaban:
[Isi jawaban di sini]

3. **Eksperimen Urutan Penguncian yang Keliru (*deadlock*).**
Kembalikan seluruh kode ke kondisi semula sehingga keluaran (*output*) kembali konsisten sebelum melanjutkan eksperimen ini.

Pada fungsi `producer()` di `kernel.c`, tukar urutan dua baris berikut sehingga `lock_acquire()` dipanggil sebelum `sema_down()`:

```
/* Urutan yang benar: */
sema_down(&sema_empty);   // (1) periksa ketersediaan slot
lock_acquire(&buf_lock);  // (2) kunci buffer

/* Urutan yang diubah untuk eksperimen ini: */
lock_acquire(&buf_lock);  // (1) kunci buffer terlebih dahulu
sema_down(&sema_empty);   // (2) periksa ketersediaan slot

```

Kode 1.11: Perbandingan urutan yang benar dan yang diubah

Kompilasi ulang dan jalankan.

```
make clean && make DEMO=-DSYNCHRONIZATION && make run

```

Kode 1.12: Kompilasi untuk eksperimen *deadlock*

(a) Tempelkan tangkapan layar (*screenshot*) layar QEMU setelah perubahan diterapkan. Catat apakah sistem menghasilkan keluaran (*output*) atau langsung berhenti.

[Tempel screenshot layar di sini]

Gambar 1.5: Tampilan QEMU pada kondisi *deadlock* akibat urutan penguncian yang keliru.

(b) Lengkapi tabel alur eksekusi berikut untuk menjelaskan mengapa perubahan urutan tersebut menyebabkan sistem berhenti total.

Tabel 1.8: Alur eksekusi akibat urutan penguncian yang keliru

Langkah	Thread Aktif	Yang Terjadi
1	*Producer*	`lock_acquire(&buf_lock)` berhasil; *Producer* memegang kunci.
2	*Producer*	`sema_down(&sema_empty)` dipanggil; nilai `sema_empty` = 0, *Producer* diblokir melalui `thread_block()`.
3	*Consumer*	Mendapat giliran CPU; `sema_down(&sema_full)` berhasil.
4	*Consumer*	`lock_acquire(&buf_lock)` dipanggil — *[Isi jawaban di sini]*
5	—	*[Jelaskan mengapa tidak ada thread yang dapat melanjutkan]*

Jawaban Singkat:

[Jelaskan kondisi yang terjadi dalam 2–3 kalimat]

(c) Apa kesimpulan yang dapat diambil terkait pentingnya urutan pemanggilan `sema_down()` dan `lock_acquire()` dalam implementasi sinkronisasi?

Jawaban:

[Isi jawaban di sini]